

Deep Learning (CAI-466)

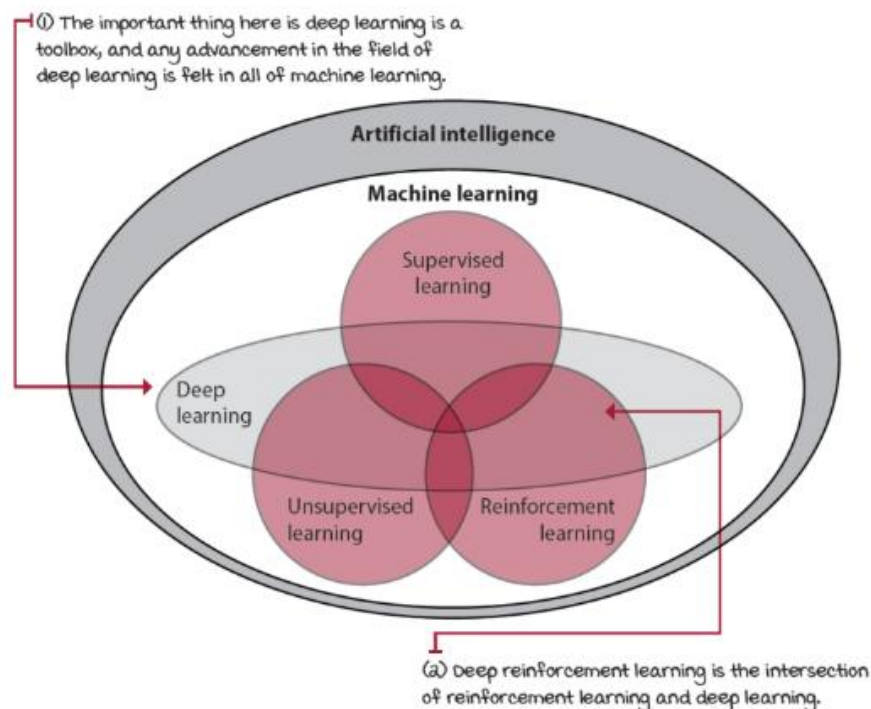
Objectives:

- Deep Reinforcement Learning

Deep Reinforcement Learning

Deep reinforcement learning (DRL) is a machine learning approach to artificial intelligence concerned with creating computer programs that can solve problems requiring intelligence. The distinct property of DRL programs is learning through trial and error from feedback that's simultaneously sequential, evaluative, and sampled by leveraging powerful non-linear function approximation.

Deep learning is a powerful toolbox

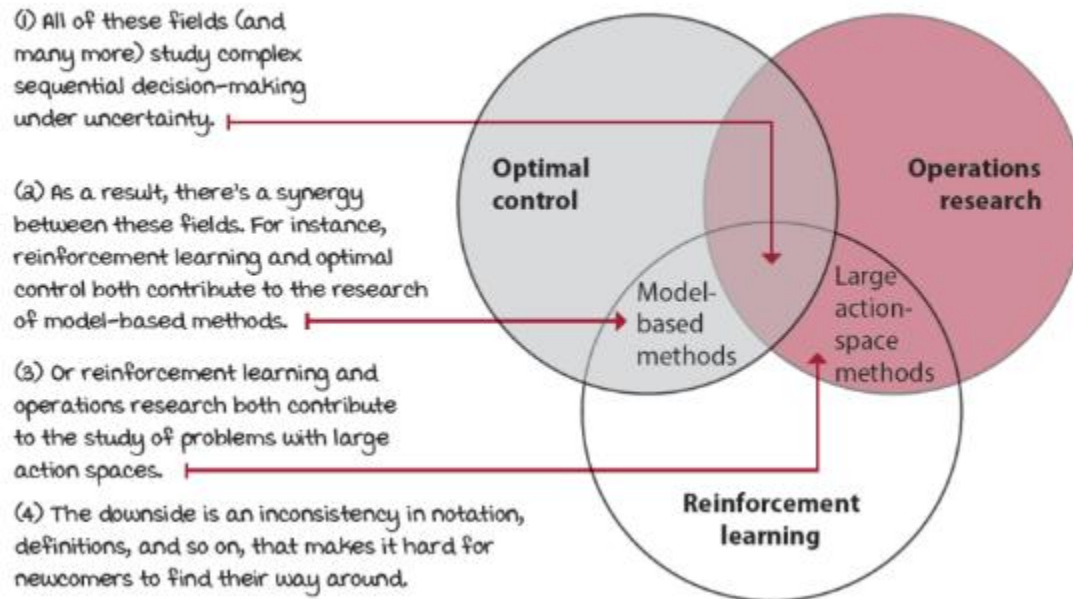


Deep reinforcement learning is concerned with creating computer programs

At its core, DRL is about complex sequential decision-making problems under uncertainty. But, this is a topic of interest in many fields; for instance, *control theory* (CT) studies ways to control

complex known dynamic systems. In CT, the dynamics of the systems we try to control are usually known in advance. *Operations research* (OR), another instance, also studies decision-making under uncertainty, but problems in this field often have much larger action spaces than those commonly seen in DRL. *Psychology* studies human behavior, which is partly the same “complex sequential decision-making under uncertainty” problem.

The synergy between similar fields



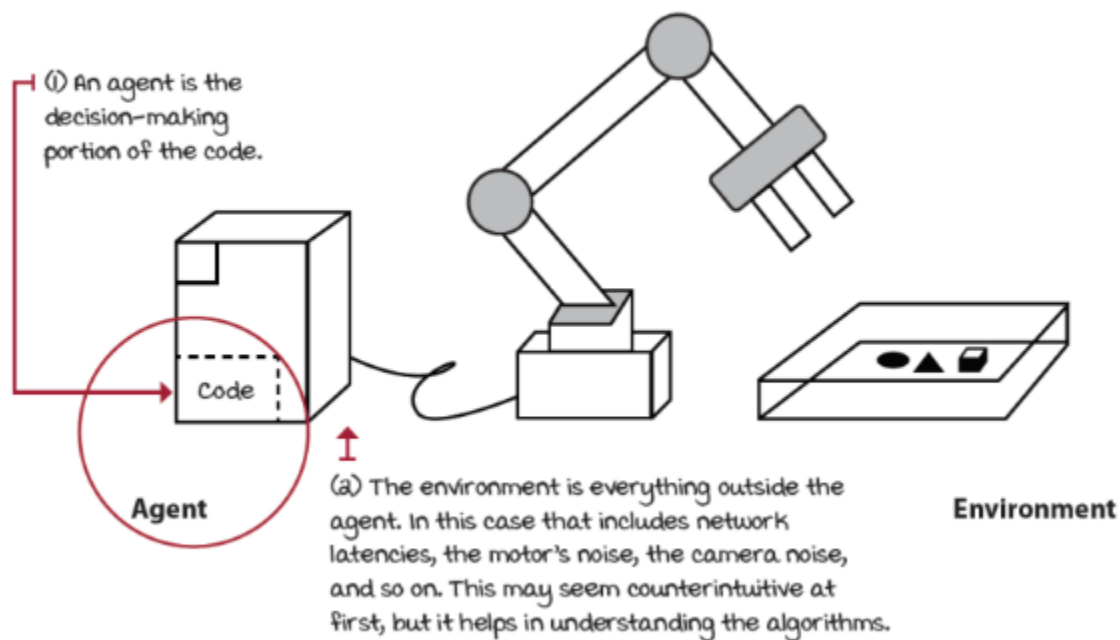
The bottom line is that you have come to a field that's influenced by a variety of others. Although this is a good thing, it also brings inconsistencies in terminologies, notations, and so on. My take is the computer science approach to this problem, so DRL is about building computer programs that solve complex decision-making problems under uncertainty, and these computer programs are called agents. An agent is a decision maker *Only* and nothing else. That means if you're training a robot to pick up objects, the robot arm isn't part of the agent. Only the code that makes decisions is referred to as the agent.

Deep reinforcement learning agents can solve problems that require intelligence

On the other side of the agent is the environment. The environment is everything outside the agent; everything the agent has no total control over. Again, imagine you're training a robot to pick up objects. The objects to be picked up, the tray where the objects lay, the wind, and

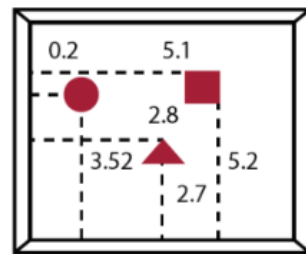
everything outside the decision maker are part of the environment. That means the robot arm is also part of the environment because it isn't part of the agent. And even though the agent can decide to move the arm, the actual arm movement is noisy, and thus the arm is part of the environment.

Boundary between agent and environment



The environment is represented by a set of variables related to the problem. For instance, in the robotic arm example, the location and velocities of the arm would be part of the variables that make up the environment. This set of variables and all the possible values that they can take are referred to as the state space. A state is an instantiation of the state space, a set of values the variables take. Interestingly, often, agents don't have access to the actual full state of the environment. The part of a state that the agent can observe is called an Observation. Observations depend on states but are what the agent can see. For instance, in the robotic arm example, the agent may only have access to camera images. While an exact location of each object exists, the agent doesn't have access to this specific state. Instead, the observations the agent perceives are derived from the states.

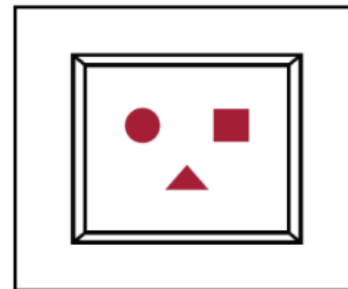
States vs. observations

**State:**
true locations

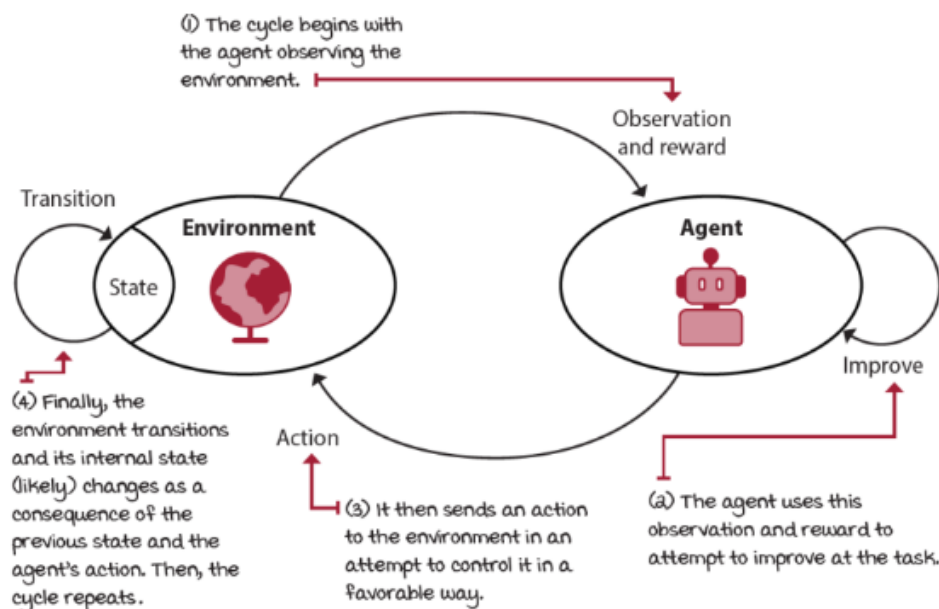
(i) States are the perfect and complete information related to the task at hand.

Observation:
just an image

(a) while observations are the information the agent receives, this could be noisy or incomplete information.



At each state, the environment makes available a set of actions the agent can choose from. The agent influences the environment through these actions. The environment may change states as a response to the agent's action. The function that's responsible for this mapping is called the **transition function**. The environment may also provide a reward signal as a response. The function responsible for this mapping is called the **reward function**. The set of transition and reward functions is referred to as the **model** of the environment.

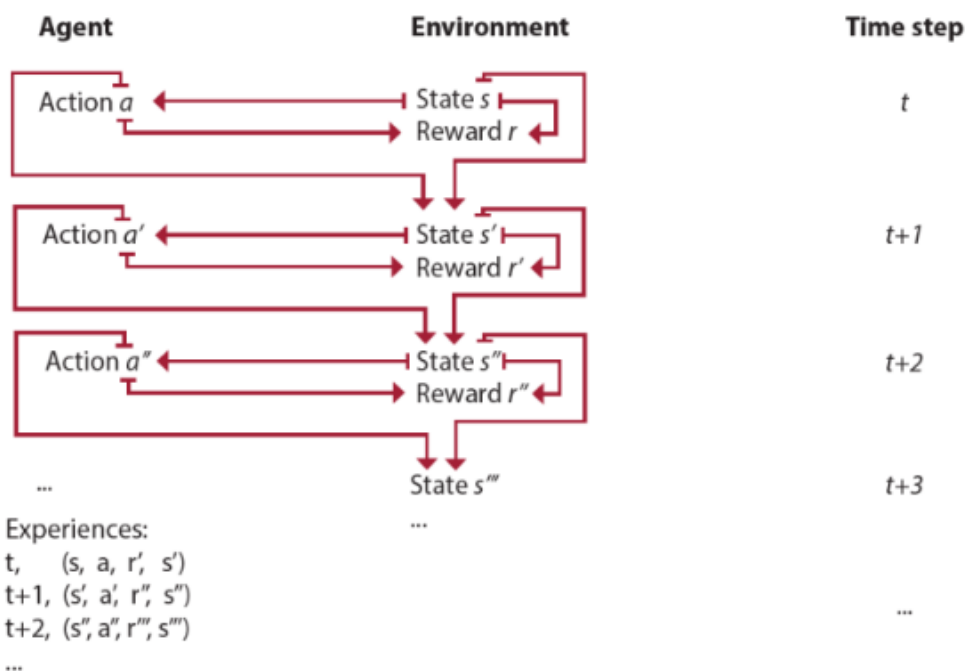
The reinforcement learning cycle

The environment commonly has a well-defined task. The goal of this task is defined through the reward function. The reward-function signals can be simultaneously sequential, evaluative, and sampled. To achieve the goal, the agent needs to demonstrate intelligence, or at least cognitive abilities commonly associated with intelligence, such as long-term thinking, information gathering, and generalization.

The agent has a three-step process: the agent interacts with the environment, the agent evaluates its behavior, and the agent improves its responses. The agent can be designed to learn mappings from observations to actions called **policies**. The agent can be designed to learn the model of the environment on mappings called **models**. The agent can be designed to learn to estimate the reward-to-go on mappings called **value functions**.

Deep reinforcement learning agents improve their behavior through trial-and-error learning

Experience tuples



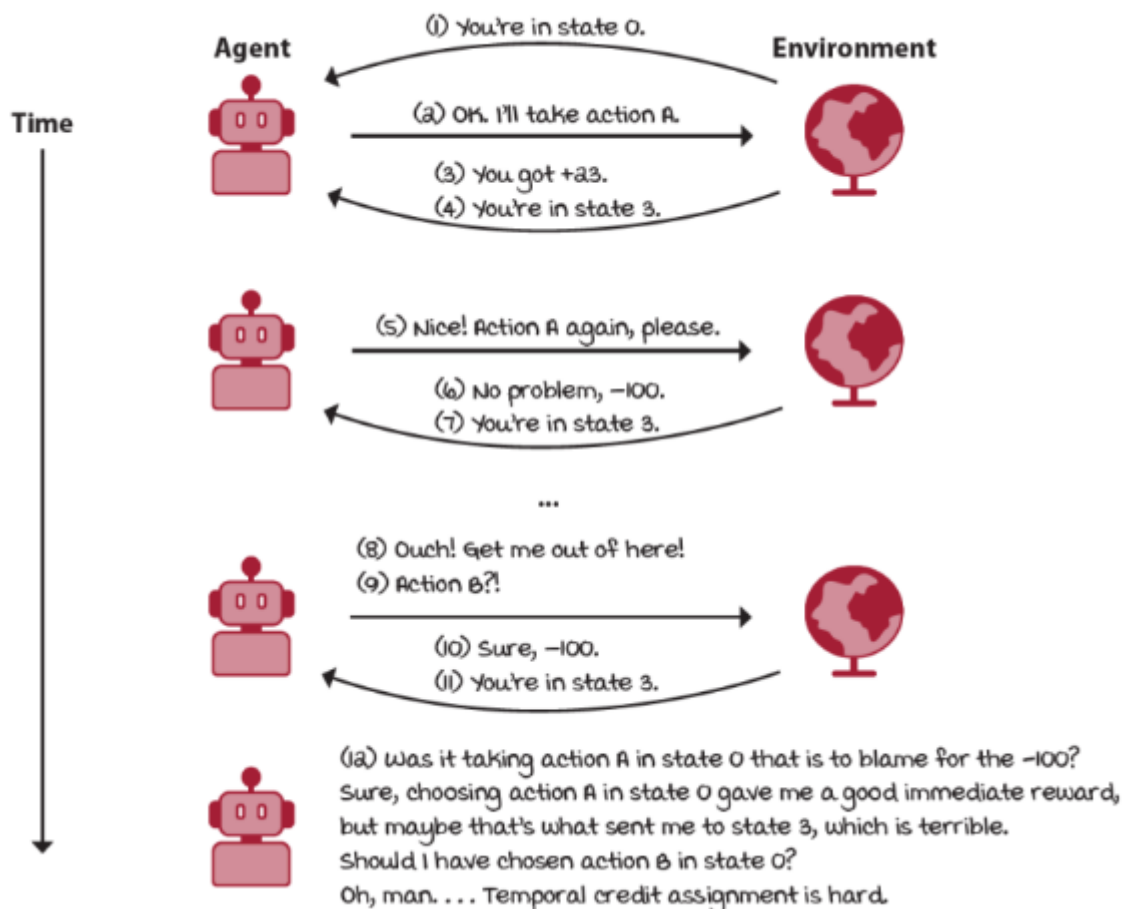
The interactions between the agent and the environment go on for several cycles. Each cycle is called a **time step**. At each time step, the agent observes the environment, takes action, and receives a new observation and reward. The set of the state, the action, the reward, and the new

state is called an **experience**. Every experience has an opportunity for learning and improving performance.

The task the agent is trying to solve may or may not have a natural ending. Tasks that have a natural ending, such as a game, are called **episodic tasks**. Conversely, tasks that don't are called **continuing tasks**, such as learning forward motion. The sequence of time steps from the beginning to the end of an episodic task is called an **episode**. Agents may take several time steps and episodes to learn to solve a task. Agents learn through trial and error: they try something, observe, learn, try something else, and so on.

Deep reinforcement learning agents learn from sequential feedback

The difficulty of the temporal credit assignment problem

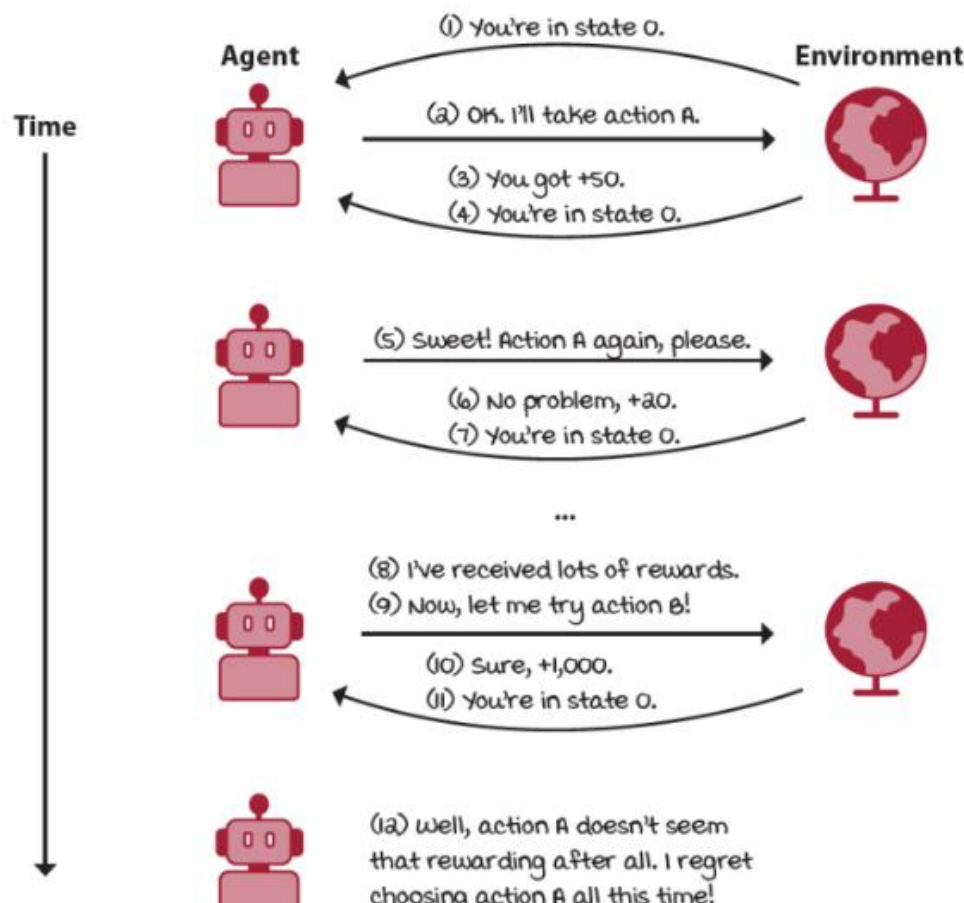


The action taken by the agent may have delayed consequences. The reward may be sparse and only manifest after several time steps. Thus the agent must be able to learn from sequential feedback. Sequential feedback gives rise to a problem referred to as the *temporal credit*

assignment problem. The temporal credit assignment problem is the challenge of determining which state and/or action is responsible for a reward. When there's a temporal component to a problem, and actions have delayed consequences, it's challenging to assign credit for rewards.

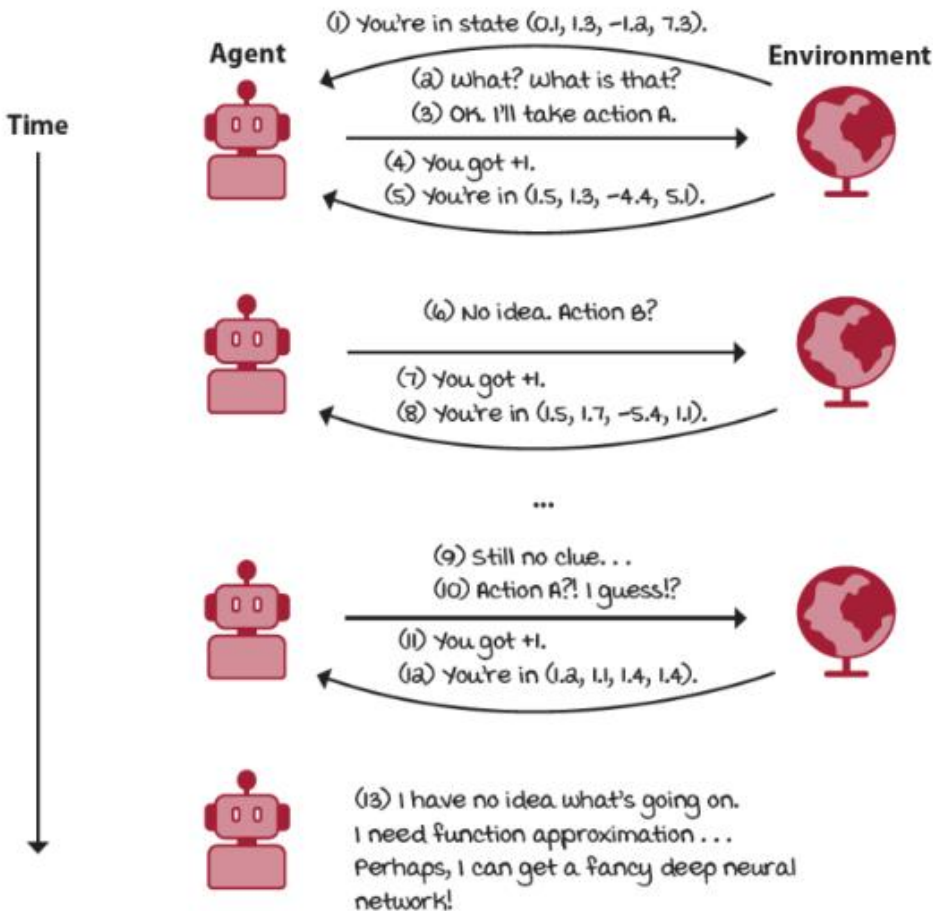
Deep reinforcement learning agents learn from evaluative feedback

The reward received by the agent may be weak, in the sense that it may provide no supervision. The reward may indicate goodness and not correctness, meaning it may contain no information about other potential rewards. Thus the agent must be able to learn from evaluative feedback. Evaluative feedback gives rise to the need for exploration. The agent must be able to balance the gathering of information with the exploitation of current information. This is also referred to as the exploration versus exploitation trade-off.



Deep reinforcement learning agents learn from sampled feedback

The reward received by the agent is merely a sample, and the agent doesn't have access to the reward function. Also, the state and action spaces are commonly large, even infinite, so trying to learn from sparse and weak feedback becomes a harder challenge with samples. Therefore, the agent must be able to learn from sampled feedback, and it must be able to generalize.



Agents that are designed to approximate policies are called *policy-based*; agents that are designed to approximate value functions are called *value-based*; agents that are designed to approximate models are called *model-based*; and agents that are designed to approximate both policies and value functions are called *actor-critic*. Agents can be designed to approximate one or more of these components.

Deep reinforcement learning agents use powerful non-linear function approximation

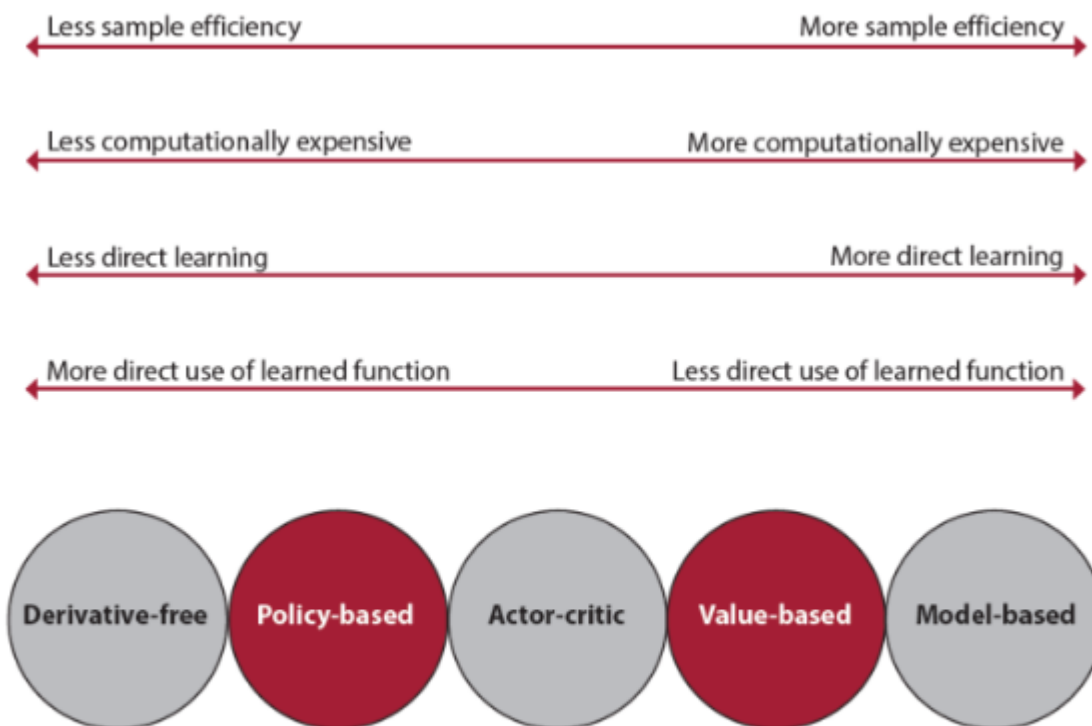
The agent can approximate functions using a variety of ML methods and techniques, from decision trees to SVMs to neural networks. However, in this book, we use only neural networks;

this is what the “deep” part of DRL refers to, after all. Neural networks aren’t necessarily the best solution to every problem; neural networks are data hungry and challenging to interpret, and you must keep these facts in mind. However, neural networks are among the most potent function approximations available, and their performance is often the best

Deep reinforcement learning is challenging because agents must learn from feedback that is simultaneously sequential, evaluative, and sampled. Learning from sequential feedback forces the agent to learn how to balance immediate and long-term goals. Learning from evaluative feedback makes the agent learn to balance the gathering and utilization of information. Learning from sampled feedback forces the agent to generalize from old to new experiences.

Algorithms used in Deep Reinforcement Learning

Comparison of different algorithmic approaches to deep reinforcement learning



↑ (i) In this book you learn about all these algorithmic approaches to deep reinforcement learning. In fact, to me, the algorithms are the focus and not so much the problems. Why? Because in DRL, once you know the algorithm, you can apply that same algorithm to similar problems with hyperparameter tuning. Learning the algorithms is where you get the most out of your time.